Defiende el servidor: versionar, comparar y recuperar un clasificador de ataques

Lis Alejandra Sánchez Moreno · Grupo B (4:00 p. m. a 6:00 p. m.) · MLOps 2026-II · Universidad Externado de Colombia
Código: github.com/Lis-Sanchez-DS/RetoDefenderServidor (público) · Modelos: huggingface.co/LisSanchez-DS/defender-servidor (privado) · Entrega: 30 de octubre de 2026

1. Datos y limpieza

Usamos las particiones publicadas de UNSW-NB15 (descarga manual de la página de UNSW el 5 de octubre de 2026). Sus SHA-256 coinciden con los del anexo del taller, y las filas y columnas coinciden con lo publicado (175.341 y 82.332 filas, 45 columnas). El objetivo es label (0 normal, 1 ataque, que es la clase positiva). Vemos como la mayoría de las filas son ataques y como hay muchísimas copias exactas de las entradas:

Archivo	Filas	Ataques	Copias de otra fila (por entradas)	Grupos con etiquetas contradictorias
training-set	175.341	68,1 %	74.301 (42,4 %)	229 (940 filas)
testing-set	82.332	55,1 %	28.386 (34,5 %)	6 (14 filas)

Política de limpieza. Las 42 variables de entrada excluyen label, attack_cat (revela la respuesta) e id (solo identifica filas, lo usamos para auditar y para el row_id de las predicciones). No hay faltantes. Los duplicados no se eliminan: cada vector de entrada idéntico forma un grupo que va entero a entrenamiento o a validación, y los grupos contradictorios se conservan (son error irreducible). Se excluyó una fila de entrenamiento con un estado raro (no); en la prueba no se eliminó ninguna fila, y los estados que el entrenamiento no vio se codifican con ceros. El preprocesamiento (estandarización, one-hot de service y state, y proto con una categoría para lo infrecuente) se ajusta solo con entrenamiento y viaja dentro del pipeline guardado.

Partición. Creamos entrenamiento y validación solo desde el archivo de entrenamiento, con semilla 42: 122.735 y 52.605 filas (70/30), con 68,06 % de ataques en ambas y 0 grupos repartidos. Es la misma para todos los experimentos. De la prueba (que solo se usó al final) medimos el solapamiento: 8.541 filas (10,4 %) tienen una entrada idéntica en entrenamiento, así que sus cifras serán algo optimistas en esas filas.

2. Cuatro experimentos, dos familias

Las hipótesis las escribí antes de entrenar (están en docs/taller_hipotesis.md, con commit previo a cada ejecución). Todas las configuraciones usan la misma partición y las mismas métricas, con el ataque como clase positiva. Para los denominadores cero, precisión y recall usan zero_division = 0 y la tasa de falsos positivos (FPR) es FP/(FP+TN), que nunca es 0/0 porque siempre hay filas normales. La línea base (C0) predice siempre la clase dominante, así que alarma todo el tráfico normal. Todas se comparan con umbral 0,5, salvo C4, que trae el suyo.

ID	Familia y cambio	Hipótesis (resumen)	Umbral	Recall	Prec.	FPR	F1 macro	Veredicto
C0	Línea base (siempre ataque)	Referencia	0,5	1,000	0,681	1,000	0,405	n/a
C1	Regresión logística, numéricas y categóricas	Supera con claridad a la línea base y deja de alarmar todo el tráfico normal	0,5	0,988	0,922	0,179	0,921	Se cumple
C2	XGBoost estándar (300 árboles, prof. 6, tasa 0,1)	Al enfocarse en la precisión, XGBoost supera a la regresión logística	0,5	0,975	0,961	0,085	0,949	Se cumple
C3	XGBoost con Optuna (25 pruebas, solo entrenamiento)	Mantiene recall >= 0,95 y reduce de forma significativa los falsos positivos	0,5	0,973	0,961	0,083	0,948	En parte
C4	XGBoost con Optuna que busca también el umbral (25 pruebas)	Con el mejor umbral balancea mejor FP y FN y logra mayor precisión	0,6607	0,954	0,974	0,055	0,944	En parte

Interpretación. Pasar de la regresión logística a XGBoost (C1 a C2) es el salto grande: las falsas alarmas bajan de 3.006 a 1.423 sobre 16.801 filas normales, aunque a 0,5 XGBoost pierde más ataques (903 frente a 443). Optuna a 0,5 (C3) casi no cambia nada frente a C2 (precisión 0,9613 contra 0,9608, 1.401 contra 1.423 falsas alarmas y 979 contra 903 ataques perdidos), por eso la hipótesis se cumple

solo en parte. C4 sube la precisión a 0,9738 con un umbral de 0,6607, pero el recall baja a 0,9537: balancea los errores en otra dirección (menos falsas alarmas, más ataques perdidos), y por eso también la marcamos como parcial. La búsqueda de C4 podó 16 de 25 pruebas (9 completas), así que exploró poco; con otra semilla el resultado podría variar.

3. Cambios entre v1 y v2

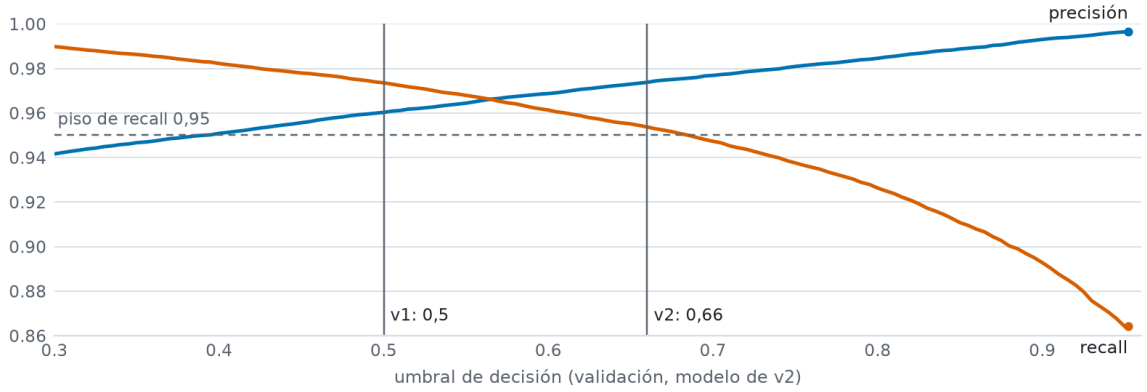
v1 es C2 (XGBoost estándar, umbral 0,50) y v2 es C4 (XGBoost de la búsqueda de Optuna, umbral 0,66, que es el 0,6607 hallado por la búsqueda y redondeado por mí). Se distinguen en los hiperparámetros y en el umbral, y sus archivos de modelo son distintos. Mi hipótesis era que Optuna lograría un mejor desempeño que el XGBoost base y se cumple en validación:

Validación (52.605 filas)	Precisión	Recall	FPR	F1 macro	Falsas alarmas (FP)	Ataques perdidos (FN)
v1 (C2, umbral 0,50)	0,9608	0,9748	0,0847	0,9487	1.423	903
v2 (C4, umbral 0,66)	0,9738	0,9537	0,0548	0,9443	920	1.656

Qué mejora y qué empeora. Mejoran la precisión (+0,0129) y las falsas alarmas (1.423 a 920). Empeoran el recall (0,9748 a 0,9537), el F1 macro y, sobre todo, los ataques perdidos (903 a 1.656). Aceptamos esa pérdida porque el recall se mantiene sobre el piso de 0,95, aunque reconozco que contradice un poco la prioridad de no perder ataques. **Hallazgo importante:** la mejora viene del umbral, no de los hiperparámetros. Con umbral 0,5 el modelo de v2 da una precisión de 0,9603, prácticamente igual que la de v1 (0,9608); y el modelo de v1 con su propio umbral de 0,6575 (recall 0,9503) alcanza precisión 0,9765 con 819 falsas alarmas, algo mejor que v2. Es decir, la búsqueda de Optuna no aportó una ventaja medible aquí; la comparación v1/v2 se hizo en validación, como pide el taller.

4. Elección del umbral

El umbral de cada versión se eligió usando solo validación. Prioricé el recall (un ataque perdido cuesta más que una falsa alarma) con un piso de 0,95, y entre los umbrales que lo cumplen busqué la mayor precisión. La figura muestra el modelo de v2 en validación: al subir el umbral la precisión sube y el recall baja, y el recall cruza el piso de 0,95 en 0,685, de modo que 0,66 queda un poco antes del cruce, con algo de margen sobre el piso (recall 0,9537). v1 se deja en 0,5 (primera solución funcional, sin ajuste). La selección de v1 y v2 se congeló en el commit 4fda613, antes de que existieran las predicciones de la prueba.



5. Prueba final

Evaluamos una sola vez v2 (modelo de SHA-256 que coincide con el congelado) sobre las 82.332 filas de la prueba, sin eliminar ninguna, con umbral 0,66. El archivo predicciones-test.csv tiene una fila por registro (row_id, y_true, score_attack, y_pred, con la regla y_pred = 1 si el puntaje es al menos el umbral) y las métricas se recalculan desde él (hay una prueba automática).

	Recall	Precisión	FPR	F1 macro	TN	FP	FN	TP
Validación	0,9537	0,9738	0,0548	0,9443	15.881	920	1.656	34.148
Prueba	0,9649	0,8596	0,1931	0,8908	29.856	7.144	1.589	43.743

Vemos como el recall se sostiene (0,965), pero la precisión cae unos 11 puntos y la FPR se multiplica por 3,5: casi una de cada cinco filas normales se marca como ataque. Por familia, Fuzzers es la más débil (recall 0,771) y Analysis le sigue (0,947); las demás están por encima de 0,97. No demostramos la causa de la caída; una hipótesis (sin comprobar) es que la prueba incluye una sesión de captura ausente del entrenamiento. El resultado se conserva aunque empeore, como pide el taller.

6. Limitaciones, trazabilidad y declaración de ayuda

Limitaciones. (a) Es un benchmark histórico de laboratorio (2015) y no demuestra eficacia en una red actual. (b) La validación estima rendimiento en distribución, con filas parecidas a ambos lados, así que es optimista. (c) Esta misma prueba ya se había mirado una vez, con otro modelo, en una versión anterior del trabajo (otra guía); no se usó para elegir el modelo ni el umbral actuales, pero conviene decirlo. (d) Solo comparamos dos familias, y la mejora de v2 no depende de los hiperparámetros. (e) Las tarjetas inmutables de v1 y v2 en Hugging Face traen por error los tamaños de una partición 80/20 descartada; la tarjeta principal tiene una fe de erratas y los modelos no cambian.

	Código (tag)	Run MLflow	Revisión de Hugging Face (SHA)	SHA-256 del modelo
v1	v1 (f40c9cc)	18fbcc33e83041e7ad99c7937583159d	053f3e62f144e78e055533f82bce14c9b54e3030	7fef2e3afa957bd67cd48fe5…
v2	v2 (f235480)	6d7cb635a3104e838d81b205f2d51b49	c521d5ccf41ad404998e3364c685363ffa620bbb	e7f7ddc749f2a22c66007031…

Recuperación. En una carpeta y un entorno nuevos (dependencias fijadas, sin cachés), recuperamos con DVC los datos originales y preparados (huellas iguales a las del anexo) y descargamos v1 y v2 de Hugging Face por su SHA completo. Con 50 filas fijas de validación, las predicciones coincidieron con las guardadas y la diferencia máxima de probabilidad fue 0.0 (tolerancia declarada 1e-09): aprobada.

Uso de IA. Usé Claude Code (Anthropic, modelo Claude Sonnet 5.5) para escribir y refactorizar el código y las pruebas, redactar la documentación y las tarjetas de modelo, ejecutar la prueba de recuperación y redactar este informe a partir de los archivos de evidencias/ y de mi perfil de estilo de escritura. Las decisiones (partición, configuraciones, definición de v1 y v2, umbrales), todas las hipótesis y sus veredictos son míos. Revisé el informe y verifiqué sus cifras contra los archivos de evidencia (una prueba automática compara las cifras del PDF con ellos). Los detalles están en entrega.md.

Referencias

Moustafa, N. y Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS, IEEE.

Moustafa, N. y Slay, J. (2016). The evaluation of Network Anomaly Detection Systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. Information Security Journal: A Global Perspective, 25(1-3), 18-31.

Chen, T. y Guestrin, C. (2016). XGBoost: a scalable tree boosting system. KDD.

Akiba, T. et al. (2019). Optuna: a next-generation hyperparameter optimization framework. KDD.

Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. JMLR, 12, 2825-2830.

Documentación de Git, DVC, MLflow Tracking y Hugging Face Hub (descarga por revisión y tarjetas de modelo).